

LiteWing Drone Control + Video — Architecture Explainer

This document explains **why** the app is built the way it is, not just what each piece does. Each section ends with the objection someone is likely to raise, and the answer.

1. The mental model: four things running at once

Almost every design decision in this app exists to manage the interaction between these four concurrent contexts:

#	What	Where it runs	Speed
1	GTK/GLib main loop	Main thread	Event-driven
2	cflib's telemetry/connection thread	Background thread (cflib creates it)	~10Hz
3	<code>persistent_stream</code> (setpoint keepalive)	Background thread (we create it)	10Hz (100ms sleep)
4	GStreamer's video pipeline	Its own thread(s)	~15-30fps

Keep this table in mind — nearly every bug encountered while building this was really "two of these four didn't coordinate correctly."

2. Why GTK, not Tkinter

The original prototype was Tkinter. We moved to GTK because of one specific capability: `xvimagesink` + `set_window_handle(xid)`, GStreamer's video-overlay API, which needs a raw X11 window handle to embed video directly into. GTK exposes this cleanly; Tkinter effectively does not.

Without that hook, Tkinter's only option is `appsink → PIL.Image → ImageTk.PhotoImage` — every video frame gets copied and converted **in Python** before display. That's real, measurable latency on every single frame.

```
python

sink = pipeline.get_by_name('sink')
xid = self.video_area.get_window().get_xid()
sink.set_window_handle(xid)
```

This is the entire reason video is fast: GStreamer writes decoded video **directly into the X server's window backing store**. Python never touches a pixel.

"Why not just use OpenCV/appsink like most tutorials?" Because that's exactly the slow path being avoided. It's simpler to write and fine for prototyping, but every frame pays a Python round-trip cost that direct X11 embedding skips entirely.

3. The video pipeline, element by element

```
python

pipe_str = (
    f'souphttpsrc location="{CAM_URL}" is-live=true do-timestamp=true '
    f'! multipartdemux '
    f'! image/jpeg '
    f'! jpegdec '
    f'! videoconvert '
    f'! queue leaky=downstream max-size-buffers=1 max-size-bytes=0 max-size-tim
    f'! xvimagesink name=sink sync=false'
)
```

Read left to right as a chain of transformations on the data:

- `souphttpsrc` — fetches the ESP32-CAM's HTTP MJPEG stream. `is-live=true` tells GStreamer not to buffer this like a file with a known duration — it's live and never-ending. `do-timestamp=true` stamps each buffer with arrival time, since this HTTP source doesn't supply reliable timestamps of its own.
- `multipartdemux` — an MJPEG-over-HTTP stream is a sequence of separate JPEG images wrapped in multipart HTTP boundaries (like email attachments). This splits the stream back into individual JPEG frames.
- `jpegdec` — decodes each JPEG into raw pixels.
- `videoconvert` — normalizes pixel format for whatever's next.
- `queue leaky=downstream max-size-buffers=1` — **the latency-creep fix**. Without it, if the sink is ever a fraction slower than the source for a moment, frames silently queue up and video drifts further behind real time over a session. Capping the queue at one frame with `leaky=downstream` means a newer frame arriving just drops the old one instead of piling up — you always see the newest frame, with lag that never grows over time.
- `xvimagesink sync=false` — the display. `sync=false` means "don't wait to pace frames to a clock, show them as they arrive" — correct for a live feed where immediacy matters more than smooth timing.

"Why not just `autovideosink`?" It picks whatever sink GStreamer thinks is best for the system, which might not support `set_window_handle()` embedding — it could open its own separate floating window instead of living inside the app.

4. The GIL lesson (the most important thing to be able to explain)

This is the actual reason the camera used to freeze the moment the drone connected — worth understanding deeply.

The mistake

The tilt-reference marks (horizon, roll arc, etc.) were originally drawn directly onto video frames using GStreamer's `cairooverlay` element — a Python callback running **inside the video pipeline itself**, once per frame:

```
python

tiltoverlay.connect('draw', self._on_overlay_draw)
```

That callback does real work every frame — trigonometry, several line strokes, text rendering — in Python, on GStreamer's own thread. Every call must acquire the **GIL** (Global Interpreter Lock — Python's rule that only one thread executes Python bytecode at a time, regardless of CPU core count).

Before the drone connects, nothing else wants the GIL much, so this is invisible. The moment the drone connects: `cflib`'s thread is pulling telemetry, `persistent_stream` is sending setpoints every 100ms, and GUI updates are firing — four threads now fight for the same lock. The video draw callback, doing the heaviest per-call work, loses that fight often enough that frames stall in the pipeline. The ESP32-CAM's tiny web server has zero patience for a slow reader and drops the connection. That's the freeze.

The fix — two complementary pieces

1. Caching + throttling (`VideoTiltOverlayPainter`) — instead of doing the expensive drawing on every video frame, do it once every 100ms (matching the telemetry rate — no point redrawing faster than the data changes) onto a cached `cairo.ImageSurface`. Every other frame just blits that cached image — one cheap C-level call instead of dozens of Python-level ones:

```
python

if needs_regen:                                # only ~10x/second
    surface = cairo.ImageSurface(...)
    self._draw_marks(scr, ...)                  # the expensive part
    self._cache_surface = surface
cr.set_source_surface(self._cache_surface, 0, 0)
cr.paint()                                     # cheap, every frame
```

2. `AttitudeIndicator` — a separate implementation as a normal GTK widget in the side panel, redrawing on the *GUI thread* (context #1), not GStreamer's thread. Structurally immune to this GIL contention, since it never competes with `cflib`/setpoint threads for video-pipeline time.

"Why not just put the drawing in a separate process?" That's actually the *most* robust fix — a separate process means a separate GIL entirely. But video could then no longer be simply embedded as a child window via `(set_window_handle())`; it would need X11 window reparenting tricks or a second floating window positioned to look attached, which is fragile against window managers. The caching fix gets most of the benefit for a fraction of the architectural complexity.

5. `Glib.idle_add` — the thread-safety pattern used everywhere

GTK widgets can **only** be touched from the main GTK thread. Telemetry arrives on `cflib`'s own thread (context #2). Every telemetry callback follows this shape:

```
python

def _apply():
    self.roll_lbl.set_text(...)    # touches a widget
    return False
Glib.idle_add(_apply)
```

`idle_add` doesn't run `_apply` immediately — it hands the function to the main loop with "run this next time you're free." That's the only safe way to move data from a background thread into the GUI. Calling `(self.roll_lbl.set_text())` directly from `cflib`'s thread would cause intermittent crashes or corruption — the kind of bug that "works on my machine" and then randomly breaks later.

Note `(_on_overlay_draw)` (GStreamer's thread) does **not** use `idle_add` — because it never touches a GTK widget, only a Cairo surface handed to it directly. That's safe from any thread. The rule isn't "background threads are dangerous" — it's specifically "GTK widgets can only be touched from the GTK thread."

6. `Gtk.Overlay` — stacking video, panel, and buttons

```
python

overlay.add(video_box)                # base layer - fills the window
overlay.add_overlay(self.control_revealer) # panel - floats on top
overlay.add_overlay(self.menu_toggle_btn)  # button - floats on top of THAT
```

`Gtk.Overlay` stacks children in **add-order**, for both rendering *and* click-handling — last added = topmost = gets clicks first. This caused a real bug: when the toggle button was added *before* the panel, once the panel opened it covered the button, so the second click (meant to close it) hit the panel instead and never reached the button. The fix was purely reordering — add the button last, always. This kind of bug is invisible until you actually click twice, which is why it slipped through initially.

7. The safety features — what each protects against

- **Two-step arm** (`(Start)` → `(CONFIRM ARM?)` within 3s) — a single accidental click can't spin up motors.
- **E-STOP + spacebar** — sends `(send_stop_setpoint())` three times (packet-loss insurance), resets commanded values so a re-arm doesn't jump back to old throttle. Kept separate from the regular Stop button so there's one unambiguous "kill it now" action, always in the same spot, always bound to a key rather than requiring precise mouse aim.
- **`(persistent_stream)` hardened with try/except** — this thread is the control link's heartbeat (sends a setpoint every 100ms, armed or not). Before hardening, if `(send_setpoint())` ever threw (a momentary link hiccup), the thread would silently die with no indication the keepalive had stopped. Now failures log (first one, then every 10th, to avoid flooding) and the loop keeps retrying.
- **Video watchdog** (`(video_watchdog_tick)`, a `(GLib.timeout_add)` every 2s) — a pad probe counts frames arriving at the sink; if the count hasn't moved between checks, the feed is stalled, and reconnect kicks in with exponential backoff (1s→2s→4s→8s, capped, **forever** — not a "give up after 5 tries" version, which is wrong for a live flight tool).

"Why not just crash/exit on error instead of all this retry logic?" Because the user might have their hands full — literally, if flying — at the exact moment something goes wrong. The software's job is to degrade gracefully and stay informative, not disappear.

Quick-reference summary table

Decision	Why	What breaks without it
GTK over Tkinter	<code>xvimagesink</code> needs native X11 window handles	Video round-trips through Python (appsink+PIL), adds latency
<code>queue leaky=downstream</code> <code>max-size-buffers=1</code>	Caps pipeline latency	Video lag creeps upward over a session
Cached/throttled overlay draw	Cuts per-frame Python/GIL work	Camera freezes/disconnects when drone connects
Panel <code>AttitudeIndicator</code> as backup	Redraws on GUI thread, not video thread	No reliable attitude reference if video stalls
<code>Glib.idle_add</code> everywhere GTK is touched	Only the GTK thread may touch GTK widgets	Intermittent crashes/corruption
<code>Gtk.Overlay</code> add-order (button last)	Add-order = click priority	Panel intercepts clicks meant for the toggle button
Two-step arm + E-STOP	Prevent accidental motor spin-up / fast kill	Single-click arm risk; no immediate kill path
<code>persistent_stream</code> try/except	Surface link failures instead of dying silently	Keepalive can stop with zero warning
Video watchdog + infinite backoff reconnect	Detect and recover from camera stalls	Feed can freeze forever with no recovery